```
      create    app/views/support_request_mailer
      create    app/views/support_request_mailer/respond.text.erb
      create    app/views/support_request_mailer/respond.html.erb
      invoke  test_unit
      create    test/mailers/support_request_mailer_test.rb
      create    test/mailers/previews/support_request_mailer_preview.rb
```

Our mailer will look similar to the mailers we've created in the past. This is what your app/mailers/support_request_mailer.rb should look like:

rails7/depot_tc/app/mailers/support_request_mailer.rb

```
class SupportRequestMailer < ApplicationMailer

  # Subject can be set in your I18n file at config/locales/en.yml
  # with the following lookup:
  #
  #   en.support_request_mailer.respond.subject
  #

»   default from: "support@example.com"
»
»   def respond(support_request)
»     @support_request = support_request
»     mail to: @support_request.email, subject: "Re: #{@support_request.subject}"
»   end

end
```

For the views, we'll show our response and quote the user's original email. As we saw in our web view, Rails will handle rendering the rich text for us, so the HTML mail view in app/views/support_request_mailer/respond.html.erb will look fairly straightforward:

rails7/depot_tc/app/views/support_request_mailer/respond.html.erb

```
<%= @support_request.response %>
<hr>
<blockquote>
  <%= @support_request.body %>
</blockquote>
```

We also want to include a plain-text version, since not everyone wants rich text